

IPre Sueño-Vigilia y Compositional Learning

Kevin Cardemil Ulloa
kevin.cardemil@uc.cl

Contenidos

I Contenidos Preliminares

II Representaciones Latentes

III Síntesis de Programas

IV Algoritmos para Redes Neuronales

V Misceláneos

VI Síntesis

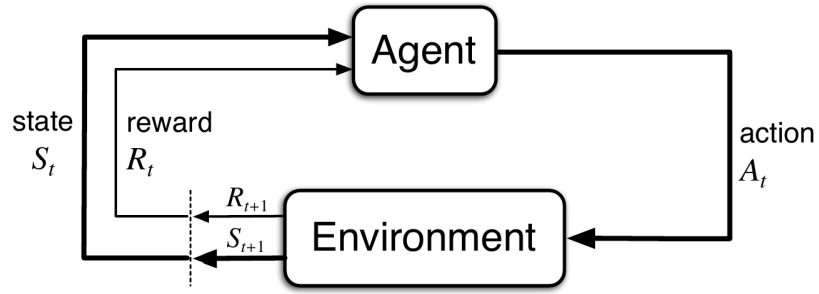
VII Apéndice

I
Contenidos Preliminares

Reinforcement Learning

Fuente *Reinforcement Learning: An Introduction*, R. S. Sutton y A. G. Barto, caps. 1 y 3.

- Un **agente** actúa, el **ambiente** responde con una **recompensa** y un nuevo **estado**.
- Aprendizaje **por interacción** guiado por retornos a largo plazo: un tercer paradigma junto al supervisado y al no supervisado.
- Componentes: **política** $\pi_t(a|s)$, **señal de recompensa**, **función de valor** y (opcional) **modelo del ambiente**.
- Tradeoff **explorar** vs. **explotar**.



Procesos de Decisión de Markov Finitos

En cada instante t: estado $S_t \in \mathcal{S}$, acción $A_t \in \mathcal{A}(S_t)$, recompensa $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$.

Las **dinámicas del ambiente** quedan determinadas por

$$p(s', r|s, a) = \Pr\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\}.$$

El agente busca maximizar el **retorno descontado** esperado, con $0 \leq \gamma \leq 1$:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

Nota

> La frontera agente-ambiente es el límite de *control* del agente, no de su conocimiento.

Complejidad de Kolmogorov

Sea $\mathcal{U}$ una máquina de Turing universal fija.

Definición I.1 Complejidad de Kolmogorov

Para $x, y \in \{0,1\}^*$, $\quad K(x|y) := \min\{|p| : \mathcal{U}(p, y) = x\}$ y $K(x) := K(x|\varepsilon)$.

Teorema I.2 Invarianza

Para máquinas universales $\mathcal{U}, \mathcal{V}$ existe $c_{\mathcal{U},\mathcal{V}} \geq 0$ tal que $|K_{\mathcal{U}}(x) - K_{\mathcal{V}}(x)| \leq c_{\mathcal{U},\mathcal{V}}$ para todo $x \in \{0,1\}^*$.

Teorema I.3

La función $x \mapsto K(x)$ es no computable.

Minimum Description Length (MDL)

Definición I.4 Minimum Description Length

Dada una clase de hipótesis $\mathcal{H}$ y datos D, se escoge

$$h^* = \arg\min_{h \in \mathcal{H}} (L(h) + L(D|h)).$$

Formaliza la **navaja de Occam**:

- Hipótesis muy simple: $L(h)$ pequeño, pero $L(D|h)$ grande.
- Hipótesis muy compleja: $L(h)$ grande, pero $L(D|h)$ pequeño (sobreajuste).

Nota

Este es el MDL «de dos partes»; el MDL «refinado» usa códigos universales.

Conceptos Bayesianos

$$\underbrace{p(\boldsymbol{z}|\boldsymbol{x})}_{\text{posterior}} = \frac{\overbrace{p(\boldsymbol{x}|\boldsymbol{z})}^{\text{verosimilitud}}\ \overbrace{p(\boldsymbol{z})}^{\text{prior}}}{\underbrace{p(\boldsymbol{x})}_{\text{evidencia}}} \qquad p(\boldsymbol{x}) = \int p(\boldsymbol{z}) p_\theta(\boldsymbol{x}|\boldsymbol{z})\, \mathrm{d}\boldsymbol{z}.$$

Máxima verosimilitud (EMV)

$$\hat{\boldsymbol{z}}_{\text{EMV}} = \arg\max_{\boldsymbol{z}} \log p(\boldsymbol{x}|\boldsymbol{z})$$

Máximo a posteriori (MAP)

$$\hat{\boldsymbol{z}}_{\text{MAP}} = \arg\max_{\boldsymbol{z}} [\log p(\boldsymbol{x}|\boldsymbol{z}) + \log p(\boldsymbol{z})]$$

Nota Posterior collapse

> Si $\boldsymbol{x}$ no informa nada, $p(\boldsymbol{x}|\boldsymbol{z}) = p(\boldsymbol{x})$ y la posterior coincide con la prior.

Variational Autoencoders

Fuente *An Introduction to Variational Autoencoders*, D. P. Kingma y M. Welling (2019). [arXiv:1906.02691]

- Queremos aproximar una distribución desconocida $p^*(\boldsymbol{x})$ con un modelo $p_{\boldsymbol{\theta}}(\boldsymbol{x}) \approx p^*(\boldsymbol{x})$.
- **Modelo profundo de variable latente** (DLVM): $p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z})$ parametrizado por una red neuronal,

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \int p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}) p_{\boldsymbol{\theta}}(\boldsymbol{z}) \, \mathrm{d}\boldsymbol{z}.$$

- **Problema:** esa integral es intratable, y por lo tanto también la posterior $p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$.
- **Solución:** un **encoder** (inferencia variacional amortizada) $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \approx p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$.

ELBO

Se cumple que

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x}) + D_{\mathrm{KL}}\big(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\big) \geq \mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x}).$$

Donde

$$\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x}) := \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\big[\log p_{\boldsymbol{\theta}}(\boldsymbol{x},\boldsymbol{z}) - \log q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})\big] \quad \textbf{ELBO}$$

La divergencia KL mide simultáneamente:

1. la distancia entre la posterior real $p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$ y su aproximación $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$;
2. el *gap* entre $\log p_{\boldsymbol{\theta}}(\boldsymbol{x})$ y la ELBO.

Por eso se maximiza $\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}$ respecto a $\boldsymbol{\theta}$ y $\boldsymbol{\phi}$ a la vez.

Truco de Reparametrización

El gradiente respecto a ϕ no entra en la esperanza, pues q_ϕ depende de ϕ. Escribimos

$$\boldsymbol{z} = g(\boldsymbol{\epsilon}, \boldsymbol{\phi}, \boldsymbol{x}), \quad \boldsymbol{\epsilon} \sim p(\boldsymbol{\epsilon}) \text{ independiente de } \boldsymbol{x}, \boldsymbol{\phi},$$

de modo que

$$\nabla_{\boldsymbol{\phi}} \mathbb{E}_{\boldsymbol{z} \sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}[f(\boldsymbol{z})] = \mathbb{E}_{\boldsymbol{\epsilon} \sim p(\boldsymbol{\epsilon})}\left[\nabla_{\boldsymbol{\phi}} f(\boldsymbol{z})\right] \simeq \nabla_{\boldsymbol{\phi}} f(\boldsymbol{z}).$$

Ejemplo I.5

$q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) = \text{Normal}(\boldsymbol{z}; \boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}))$ con $(\boldsymbol{\mu}, \log \boldsymbol{\sigma}) = \text{EncoderNeuralNet}_{\boldsymbol{\phi}}(\boldsymbol{x})$ y $\boldsymbol{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \text{Normal}(0, I)$.

Algoritmo AEVB

Algoritmo I.6 AEVB

Data: dataset $\mathcal{D}$, modelo de inferencia $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$, modelo generativo $p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z})$.

1. $(\boldsymbol{\theta}, \boldsymbol{\phi}) \leftarrow$ inicializar parámetros.
2. **while** SGD *no ha convergido* **do**:
3. | $\mathcal{M} \sim \mathcal{D}$ (minibatch) y $\boldsymbol{\epsilon} \sim p(\boldsymbol{\epsilon})$ (ruido para cada punto de $\mathcal{M}$).
4. | Calcular $\tilde{\mathcal{L}}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathcal{M}, \boldsymbol{\epsilon})$ y sus gradientes $\nabla_{\boldsymbol{\theta},\boldsymbol{\phi}} \tilde{\mathcal{L}}_{\boldsymbol{\theta},\boldsymbol{\phi}}$.
5. | Actualizar $\boldsymbol{\theta}, \boldsymbol{\phi}$ con un optimizador SGD.

Generar: $\boldsymbol{z}' \sim p_{\boldsymbol{\theta}}(\boldsymbol{z})$, luego $\boldsymbol{x} \sim p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}')$.

Reconstruir: $\boldsymbol{z}' \sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$, luego $\boldsymbol{x}' \sim p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}')$.

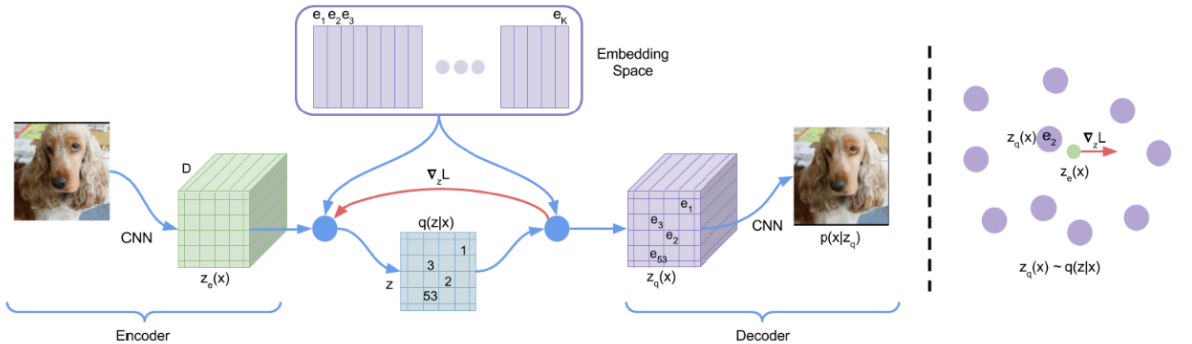
II
Representaciones Latentes

VQ-VAE

Fuente *Neural Discrete Representation Learning*, A. van den Oord et al. (2017). [arXiv:1711.00937]

Combina VAEs con **cuantización vectorial** para obtener un espacio latente **discreto**: un codebook $e_1, \ldots, e_K \in \mathbb{R}^D$ y una posterior categórica determinista

$$q(z=k|x) = \begin{cases} 1 & \text{si } k = \arg\min_j \left\| z_e(x) - e_j \right\|_2 \\ 0 & \text{e.o.c.} \end{cases} \qquad z_q(x) = e_k.$$



VQ-VAE: Aprendizaje

$$L = \underbrace{\log p\big(x|z_q(x)\big)}_{\text{reconstrucción}} + \underbrace{\|\text{sg}[z_e(x)] - e\|_2^2}_{\text{VQ}} + \underbrace{\beta\|z_e(x) - \text{sg}[e]\|_2^2}_{\text{compromiso}}.$$

- sg es el operador *stop-gradient* (identidad hacia adelante, derivada nula).
- El arg min no tiene gradiente: se **copia** el gradiente de $z_q(x)$ a $z_e(x)$.
- Robusto a $\beta \in [0.1, 2.0]$.
- Con prior uniforme, la KL es constante ($= \log K$). Al final, la prior se ajusta a una distribución **autorregresiva** (PixelCNN, WaveNet):

$$p(\boldsymbol{z}) = \prod_{t=1}^{T} p(z_t|z_1, ..., z_{t-1}).$$

Gumbel-Softmax

Fuente *Categorical Reparameterization with Gumbel-Softmax*, E. Jang et al. (2016). [arXiv:1611.01144]

Gumbel-Max trick: para muestrear z con probabilidades de clase π,

$$z = \text{one_hot}\Big(\arg\max_i [g_i + \log \pi_i]\Big), \quad g_i \sim \text{Gumbel}(0, 1).$$

Se relaja el $\arg\max$ con un softmax de **temperatura** $\tau > 0$:

$$y_i = \frac{\exp((g_i + \log \pi_i)/\tau)}{\sum_{j=1}^{k} \exp\big((g_j + \log \pi_j)/\tau\big)} \in \Delta^{k-1}.$$

Si $\tau \to 0^+$, las muestras se vuelven one-hot y se recupera la categórica.

Estimadores Gumbel-Softmax

- **Gumbel-Softmax:** reemplaza las muestras categóricas por y, que es diferenciable respecto a π, y permite *backpropagation*.
- **Tradeoff:** τ pequeño da muestras casi one-hot pero gradientes de alta varianza. En la práctica se parte con τ alto y se va reduciendo (*annealing*).
- **Straight-Through (ST):** usa $z = \text{one_hot}(\arg\max)$ en el *forward pass* y la relajación en el *backward pass*:

$$\nabla_\theta z \approx \nabla_\theta y.$$

  Sirve cuando se **necesita** una muestra discreta (p. ej., acciones en RL).

Nota

> En VAEs con variables categóricas, ambos estimadores superan a los demás estimadores comparados.

Posterior Collapse

El **colapso posterior** ocurre cuando el encoder ignora (por lo general parte de) el espacio latente:

$$q_\phi(\boldsymbol{z}|\boldsymbol{x}) = p_\theta(\boldsymbol{z}).$$

Se ha atribuido históricamente al término KL de la ELBO (de ahí propuestas como el β-VAE), pero la explicación es más fina.

Fuente *Posterior Collapse as a Phase Transition in VAEs*, Z. Li et al. (2025). [arXiv:2510.01621]

En *deep Gaussian VAEs* el colapso aparece como una **transición de fase**, aproximadamente cuando

$$\sigma'^2 > \max[\xi_1^2, ..., \xi_N^2],$$

con ξ_i^2 los valores propios de la covarianza de los datos: si hay demasiado ruido, el modelo ignora los datos.

Colapso y No-Identificabilidad

Fuente *Posterior Collapse and Latent Variable Non-identifiability*, Y. Wang et al. (2021). [arXiv:2301.00537]

Definición II.1 Variable latente no-identificable

z es **no-identificable** en $\hat{\theta}$ si $p\left(x|z=\tilde{z}';\hat{\theta}\right)=p\left(x|z=\tilde{z};\hat{\theta}\right)$ para todo $\tilde{z}',\tilde{z}\in\mathcal{Z}$.

Teorema II.2 Caracterización del colapso posterior

Las variables latentes z son no-identificables en $\hat{\theta}$ si, y sólo si, la posterior de z colapsa: $p\left(z|x;\hat{\theta}\right)=p(z)$.

Colapso y No-Identificabilidad (cont.)

Demostración

($\Longrightarrow$) Por Bayes y la no-identificabilidad,

$$p\left(\boldsymbol{z}|\boldsymbol{x};\hat{\theta}\right) \propto p(\boldsymbol{z})p\left(\boldsymbol{x}|\boldsymbol{z};\hat{\theta}\right) = p(\boldsymbol{z})p\left(\boldsymbol{x};\hat{\theta}\right) \propto p(\boldsymbol{z}).$$

($\Longleftarrow$) Si colapsa, $p(\boldsymbol{z}) = p\left(\boldsymbol{z}|\boldsymbol{x};\hat{\theta}\right) \propto p(\boldsymbol{z})p\left(\boldsymbol{x}|\boldsymbol{z};\hat{\theta}\right)$, así que $p\left(\boldsymbol{x}|\boldsymbol{z};\hat{\theta}\right)$ es constante en $\boldsymbol{z}$. ■

- La caracterización vale más allá de los VAEs (p. ej., PPCA).
- Proponen el **LIDVAE**, que usa mapeos de Brenier para tener una verosimilitud inyectiva en $\boldsymbol{z}$ y así no colapsar.

¿Por qué se dice que VQ-VAE evita el Colapso Posterior?

Con posterior one-hot y prior uniforme sobre K códigos, la KL de la ELBO es constante:

$$D_{\mathrm{KL}}(q(z|x) \parallel p(z)) = \sum_{k=1}^{K} q(z=k|x) \log \frac{q(z=k|x)}{1/K} = 1 \cdot \log \frac{1}{1/K} = \log K,$$

así que su gradiente respecto a los parámetros es nulo.

VQ-VAE sólo evita una parte del colapso

Al ser constante, la KL ya **no empuja** $q_\phi(\boldsymbol{z}|\boldsymbol{x})$ hacia la prior durante la maximización de la ELBO. Eso elimina la causa del colapso que proviene de la KL, pero no las demás: por ejemplo, un decoder capaz de ignorar $\boldsymbol{z}$ (no-identificabilidad) o datos con demasiado ruido.

III
Síntesis de Programas

DreamCoder

Fuente *DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning*, K. Ellis et al. (2021). [arXiv:2006.08381]

Dado un conjunto de tareas $\mathcal{X}$ (pares input-output) provenientes del mismo dominio, busca programas $\{\varrho_x\}$ que las resuelvan, **construyendo su propio DSL** (librería L), que es compartido entre todos los programas.

- *Retos*: explosión combinatorial, generalización e interpretabilidad.

Definición III.1

El **joint description length** de una librería L y programas $\{\varrho_x\}$ es

$$\mathrm{DL}(L, \{\varrho\}, \mathcal{X}) := |L| + \sum_{x \in \mathcal{X}} |\varrho_x|_L .$$

El objetivo es el del MDL, pero es intratable: se introduce un **modelo de reconocimiento** $Q_\theta(\varrho|x)$ que guía la búsqueda.

DreamCoder: Ciclo Wake-Sleep

Fase	Actualiza	Qué hace
Wake	$\{\varrho_x\}$	Busca, para cada tarea, el programa que maximiza $\log Q_\theta(\varrho\|x) + \log P(\varrho\|L)$ (prior sobre la librería).
Sleep: abstracción	L	$L^* = \arg\max_L \left[\log P(L) + \sum_x \log P(\varrho_x^*\|L)\right]$: comprime los programas en nuevas funciones.
Sleep: sueños	θ	Entrena Q_θ con **replays** (x, ϱ_x^*) y **fantasías** (x, ϱ) con $\varrho \sim P(\varrho\|L)$.

Nota Conexión con DreamerV3 (más adelante)

> Las fantasías cumplen un rol parecido al modelo de mundo de DreamerV3: permiten ensayar con datos que no vienen de la fuente original.

Searching Latent Program Spaces (LPN)

Fuente *Searching Latent Program Spaces*, M. V. Macfarlane et al. (2024). [arXiv:2411.08706]

Toma lo mejor de los dos enfoques para resolver ARC-AGI:

- **Inductivo** (síntesis simbólica): programa explícito y búsqueda en test-time, pero necesita un DSL hecho a mano.
- **Transductivo** (aprendizaje profundo): escalable, pero con pesos fijos tras entrenar y propenso al sobre-ajuste.

La **Latent Program Network** representa programas en un **espacio latente continuo** y hace la búsqueda en test-time **ahí**, no en los parámetros de la red.

$$\underbrace{z \sim q_\phi(z|x,y)}_{\text{Encoder}} \quad \longrightarrow \quad \underbrace{z' = f(p_\theta, z, x, y)}_{\text{Optimización latente}} \quad \longrightarrow \quad \underbrace{\hat{y} \sim p_\theta(y|x,z')}_{\text{Decoder}}$$

LPN: Optimización en el Espacio Latente

Se busca el programa latente que mejor explica los ejemplos bajo el decoder actual:

$$z' \in \arg\max_{z} \sum_{i=1}^{n} \log p_\theta(y_i | x_i, z).$$

- **Muestreo:** se toman K latentes y se escoge el de mayor log-verosimilitud. Escala mal con la dimensión.
- **Ascenso de gradiente:** se parte del promedio de los latentes del encoder,

$$z'_k = z'_{k-1} + \alpha \cdot \nabla_z \sum_{i=1}^{n} \log p_\theta(y_i | x_i, z) \Big|_{z=z'_{k-1}}.$$

Nota Sistema 1 y Sistema 2

El encoder da una intuición rápida (Sistema 1) y la optimización latente la refina deliberadamente (Sistema 2). Ninguno basta por sí solo.

LPN: Entrenamiento

Estrategia **leave-one-out**: para reconstruir y_i se codifican y promedian los pares $j \neq i$, se optimiza el latente y se evalúa en (x_i, y_i).

$$\mathcal{L}_{\text{total}}(\phi, \theta) = \underbrace{\sum_{i=1}^{n} -\log p_\theta(y_i | x_i, z_i')}_{\mathcal{L}_{\text{rec}}} + \beta \underbrace{\sum_{i=1}^{n} D_{\text{KL}}\big(q_\phi(z | x_i, y_i) \parallel \text{Normal}(0, I)\big)}_{\mathcal{L}_{\text{KL}}}.$$

- $\mathcal{L}_{\text{rec}}$ mide qué tan mal predice el decoder; $\mathcal{L}_{\text{KL}}$ mantiene suave el espacio latente.
- Se usa el truco de reparametrización $z = \mu + \epsilon \cdot \Sigma$.
- En test-time, ϕ y θ quedan fijos: sólo se mueve z.

LPN: Resultados en la Tarea Pattern

Entrenamiento	Grad 0	Grad 1	Grad 5	Grad 20	Grad 100	Sample 250
Grad 0	3.2	3.6	18.8	52.5	67.5	3.2
Grad 1	8.6	44.6	85.4	98.4	**99.5**	10.2
Grad 1 $\star\star$	0.6	13.7	60.2	88.9	94.1	0.7
Grad 5	0.0	0.4	31.9	88.5	98.1	0.5
Sample 5	6.1	8.2	27.7	56.3	72.2	6.1

Tabla 1 — Accuracy (%). Filas: entrenamiento. Columnas: inferencia.

1. Sin búsqueda en inferencia, todo falla.
2. Más búsqueda $\Rightarrow$ mejor desempeño.
3. Entrenar sabiendo que habrá búsqueda importa.
4. El gradiente supera ampliamente al muestreo.

LPN: Generalización y Limitaciones

	Grad 0	Grad 10	Grad 100
In-Context	0.0	—	—
TTT	0.0	1.8	0.3
LPN Grad 0	0.3	18.8	41.1
LPN Grad 1	0.0	59.9	**88.0**

Tabla 2 — Accuracy (%) en Pattern fuertemente OOD.

- **La búsqueda en test-time permite adaptarse a tareas nuevas.**
- **Composición limitada:** compone dos operaciones vistas por separado, pero falla con tres. Cada programa es un **único vector** z, sin «partes» reutilizables.

Neural Language Interpreter (NLI)

Fuente *Gradient-Based Program Synthesis with Neurally Interpreted Languages*, M. V. Macfarlane et al. (2026). [arXiv:2604.18907]

- **Latent Adaptation Networks (LANs):** encoder-decoder que aprende un espacio latente de programas y busca en él en test-time. LPN y NLI son LANs.
- NLI aprende de los pares input-output su **propio lenguaje discreto**, un **codebook** de K tokens «simbólicos» (no definidos por humanos): una especie de DSL aprendido.
- Un programa es una **secuencia de tokens** de largo variable, en vez de un único vector:
  - el **inductor de programa** q_ϕ (encoder) infiere el programa desde los pares;
  - el **intérprete neuronal** p_θ (decoder) lo ejecuta token a token sobre un input.
- El intérprete es **diferenciable**: eso permite buscar programas con ascenso de gradiente.
- Puede resolver problemas que requieren **más tokens** de los vistos en entrenamiento.

NLI: El Inductor q_ϕ

El inductor mapea cada par (x, y) a una secuencia de distribuciones sobre el codebook:

$$z = (z_1, ..., z_T) \in \left(\Delta^{K-1}\right)^T,$$

$$\Delta^{K-1} := \left\{ x \in \mathbb{R}^K : x_i \geq 0, \sum_{i=1}^{K} x_i = 1 \right\}.$$

Se promedian por posición los embeddings de los pares, $\bar{e}_t = \frac{1}{|S|} \sum_i e_t^{(i)}$, y se muestrea con Gumbel-Softmax:

$$z_t = \text{Softmax}\big((f_\phi(\bar{e}_t) + g_t)/\tau_e\big), \quad g_t \sim \text{Gumbel}(0, 1),$$

con la temperatura τ_e bajando durante el entrenamiento (*annealing*).

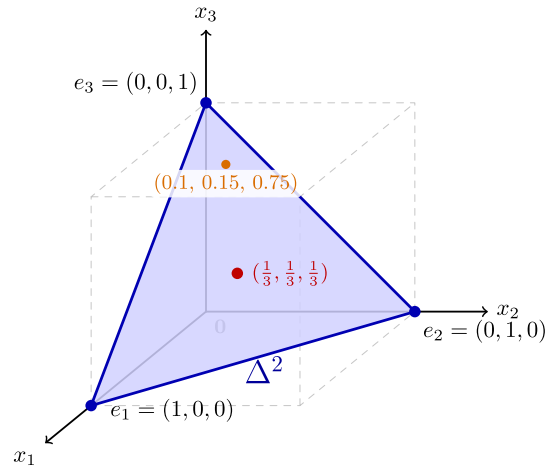


Figura 3 — El símplex Δ^2.

Nota

> Un **skip token** permite programas de largo variable aun con T fijo.

NLI: El Intérprete p_θ

El intérprete aprende dos **embedding matrices** y una red de ejecución d:

- **Code embedder** E_v: su fila k es el vector del token k; $E_v^T z_t$ «busca en el codebook» la instrucción del paso t.
- **Input-output embedder** E_{io}: su fila k representa el valor k que puede tomar una celda de los datos.

Parte de $s_0 = \text{Embed}(x)$, $\pi_0 = \text{OneHot}(x)$ y en cada paso $t = 1, \ldots, T$:

$$n_t = d(s_{t-1}, E_v^T z_t), \quad \tilde{\pi}_t = \text{Softmax}((n_t + h_t)/\tau_d), \quad h_t \sim \text{Gumbel}(0, 1),$$

$$\pi_t = z_{t,\text{skip}} \cdot \pi_{t-1} + \left(1 - z_{t,\text{skip}}\right) \cdot \tilde{\pi}_t, \quad s_t = E_{\text{io}}^T \pi_t.$$

La predicción es π_T: el **estado interno** s_t se actualiza instrucción a instrucción.

NLI: Entrenamiento

Leave-one-out sobre batches de B especificaciones con m pares, donde $\mathcal{S}_{b,i} := \mathcal{S}_b \setminus \{(x_{b,i}, y_{b,i})\}$:

$$\mathcal{L}(\phi, \theta; \mathcal{S}) = \frac{1}{B} \sum_{b=1}^{B} \left(\frac{1}{m} \sum_{i=1}^{m} \mathcal{L}_{\text{recon}}(\phi, \theta; x_{b,i}, y_{b,i}, \mathcal{S}_{b,i}) + \lambda_{\text{reg}} \cdot \mathcal{L}_{\text{reg}}(\phi; \mathcal{S}_b) \right).$$

- **Reconstrucción:** $\mathcal{L}_{\text{recon}}(\phi, \theta; x, y, S) := -\log p_\theta\big(y | x, q_\phi(S)\big)$ asegura que el lenguaje sea expresivo.
- **Regularización:** premia **reutilizar tokens** (¡composicionalidad!):

  $$\mathcal{L}_{\text{reg}}(\phi; \mathcal{S}) := \sum_{k=1}^{K} \sum_{j=1}^{T} \left[1 - \prod_{b=1}^{B} \prod_{i=1}^{m} \Big(1 - q_\phi^{(k,j)}(\mathcal{S}_{b,i})\Big) \right],$$

  el número esperado de tokens distintos usados en el batch.
- **Annealing:** $\tau_e : 8 \to 0.5$ y $\tau_d : 2 \to 0.5$. A diferencia de LPN, **no** hay bucle interno de búsqueda al entrenar.

NLI: Búsqueda en Test-Time

Se optimizan los **embeddings continuos** e del inductor, no los tokens directamente:

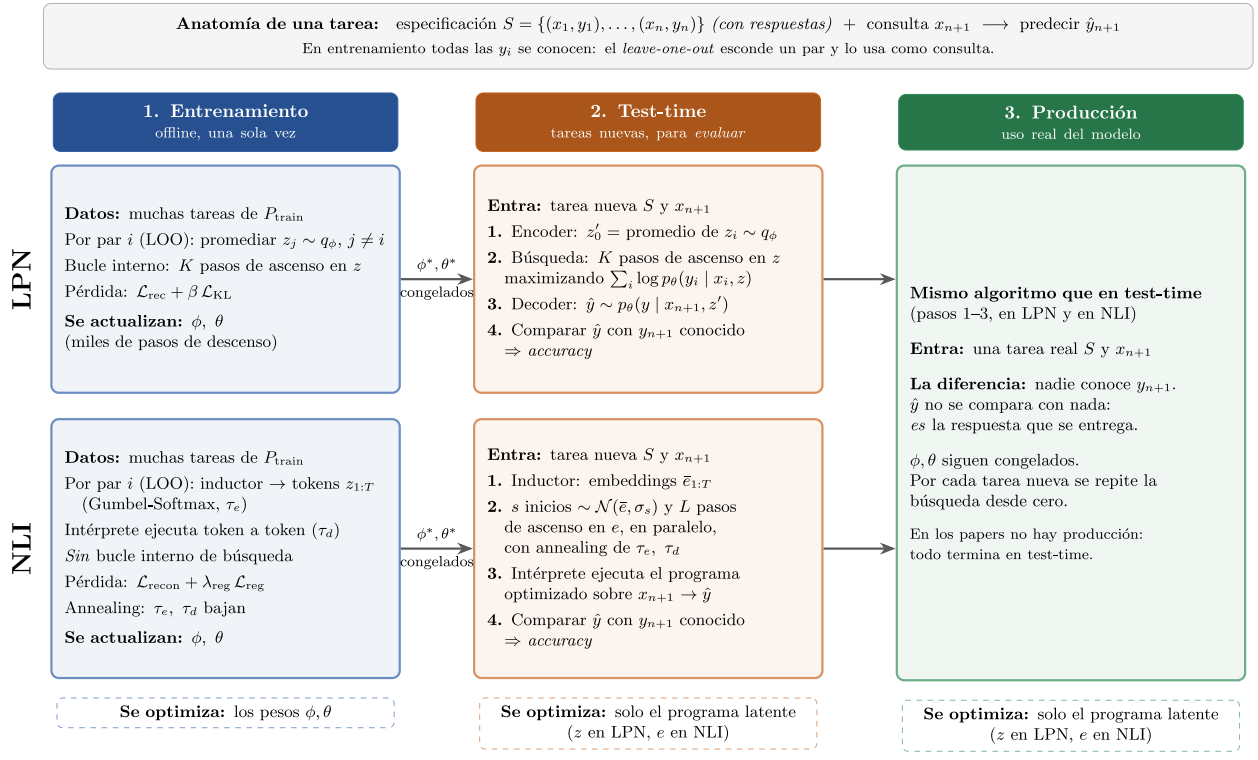
$$e^* = \arg\max_e \mathbb{E}_{g,h}\left[\sum_{(x_j,y_j)\in S} \log p_\theta\big(y_j|x_j, z(e,\tau_e,g),\tau_d,h\big)\right].$$

- Se inicializan s puntos de partida $\sim \mathcal{N}(\bar{e},\sigma_s)$ y se hacen L pasos de ascenso de gradiente **en paralelo**.
- Las temperaturas τ_e,τ_d parten **altas** (explorar) y bajan (*annealing*) hasta converger a **programas discretos**.
- Finalmente, el intérprete ejecuta el programa optimizado sobre x_{n+1}.

Nota Comparación con LPN

LPN busca en un vector continuo z; NLI busca en un espacio continuo **relajado** que, al bajar la temperatura, termina en una secuencia de tokens.

LPN vs NLI: Entrenamiento, Test-Time y Producción



El Aporte de la Discretización

LPN

- Programa = un vector continuo z.
- Ascenso de gradiente en z.
- Compone 2 operaciones; falla con 3.

NLI

- Programa = secuencia de tokens (Gumbel-Softmax con τ decreciente).
- Ascenso de gradiente sobre los embeddings e, con annealing de τ_e, τ_d.
- El intérprete actualiza un **estado interno** s_t token a token: **generalización de longitud**.

Hipótesis

El pensamiento humano funciona en «bloques»: razonamos secuencialmente componiendo *ideas* aisladas, algo que las representaciones discretas simbolizan mejor que los espacios latentes continuos.

¿Por qué no un NLI con Espacio Latente Continuo?

No se prioriza un vocabulario

Tokens continuos similares o idénticos no se pueden distinguir: no hay un conjunto finito de «palabras» que reutilizar, y $\mathcal{L}_{\text{reg}}$ pierde su sentido.

Se acumulan errores

Al evaluar en el intérprete, cada paso recibe una instrucción imprecisa y el error se propaga y crece a lo largo de los T pasos.

Más sobreajuste

El espacio de búsqueda sería mucho más flexible: en test-time sería más fácil encontrar un programa que calce con los pocos ejemplos sin capturar la regla que los generó.

Nota

LPN es justamente el caso continuo: compone 2 operaciones, pero falla con 3.

IV
Algoritmos para Redes Neuronales

Flat Minima

Fuente *Flat Minima*, S. Hochreiter y J. Schmidhuber (1997). [DOI:10.1162/neco.1997.9.1.1]

Para una red $\mathrm{net}(w) : x \mapsto o(w; x)$ con pesos $w \in \mathbb{R}^L$:

- w es un **mínimo aceptable** si $E(\mathrm{net}(w), D_0) = \sum_p \left\| y_p - o(w; x_p) \right\|^2 \leq E_{\mathrm{tol}}$.
- Un **mínimo plano** es una región conexa de mínimos aceptables.
- Se busca un mínimo aceptable cuya **caja** M_w (hipercuboide centrado en w) tenga gran volumen.

Flat Minimum Search (FMS) minimiza

$$E(w, D_0) := E(\mathrm{net}(w), D_0) + \lambda B(w, X_0),$$

donde minimizar B maximiza el volumen de la caja.

FMS: ¿Por qué Generalizan los Mínimos Planos?

Algoritmo IV.1 FMS (un paso por ejemplo (x_p, y_p))

1. **Forward:** calcular $o(w; x_p)$.
2. **Backward:** obtener ∇E_p y $\partial o^k / \partial w_{ij}$.
3. Calcular ∇B (con el operador $\mathcal{R}$ de Pearlmutter: costo $\mathcal{O}(L)$).
4. $w \leftarrow w - \alpha \nabla E_p - \lambda \cdot \text{escala} \cdot \nabla B$.

Nota MDL

En una caja el error se mantiene acotado, así que los pesos necesitan poca precisión: pocos bits. Un error bajo corresponde a $L(D|h)$ y un B bajo a $L(h)$.

Nota

El backprop convencional prefiere las direcciones más empinadas; con batches pequeños, el ruido del SGD favorece implícitamente mínimos planos (Keskar et al., 2017). FMS lo hace explícito.

V
Misceláneos

DreamerV3

Fuente *Mastering Diverse Domains through World Models*, D. Hafner et al. (2023). [arXiv:2301.04104]

Algoritmo de RL que aprende un **modelo de mundo** y deja que el agente **imagine** el futuro. Tres redes entrenadas a la vez:

Modelo de mundo (RSSM)

- Memoria: $h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1})$.
- Encoder: $z_t \sim q_\phi(z_t|h_t, x_t)$.
- Predictor: $\hat{z}_t \sim p_\phi(\hat{z}_t|h_t)$.
- Estado: $s_t = \{h_t, z_t\}$, desde el que se predicen r_t, c_t y se reconstruye x_t.

Actor y crítico (sólo sobre s_t)

$$a_t \sim \pi_\theta(a_t|s_t), \quad v_\psi(R_t|s_t).$$

El crítico aprende los λ-*returns*

$$R_t^\lambda = r_t + \gamma c_t\big((1-\lambda)v_t + \lambda R_{t+1}^\lambda\big).$$

DreamerV3: Ciclo de Aprendizaje

1. **Actuar:** el actor juega en el ambiente real y guarda las trayectorias en el *replay buffer*.
2. **Entrenar**, en cada paso de gradiente:
   - tomar un batch de trayectorias reales y actualizar el modelo de mundo;
   - **imaginar** trayectorias desde esos estados latentes con el modelo y el actor actuales;
   - actualizar actor y crítico con las trayectorias imaginadas.
3. **Iterar.**

Los retornos se normalizan con $S = \mathrm{EMA}\big(\mathrm{Per}\big(R_t^\lambda, 95\big) - \mathrm{Per}\big(R_t^\lambda, 5\big), 0.99\big)$, así la exploración no depende de la escala de las recompensas.

Resultado

> Primer algoritmo en conseguir diamantes en Minecraft sin datos humanos, con hiperparámetros fijos entre dominios. Más parámetros $\Rightarrow$ mejor rendimiento y menos datos.

VI
Síntesis

Principios del Aprendizaje Composicional

Cuestionarlos! Puede servir la práctica para discutirlo

1. Lo simple generaliza mejor (MDL)

Una descripción corta no alcanza a memorizar el ruido: para comprimir debe capturar la regularidad real de los datos, que es la que se repite en datos nuevos. *DreamCoder, Flat Minima.*

2. Aprender de la imaginación

Se entrena con experiencia generada por el propio modelo: trayectorias imaginadas en el modelo de mundo (*DreamerV3*) y fantasías muestreadas de la librería (*DreamCoder*).

3. Intuición rápida y luego refinamiento

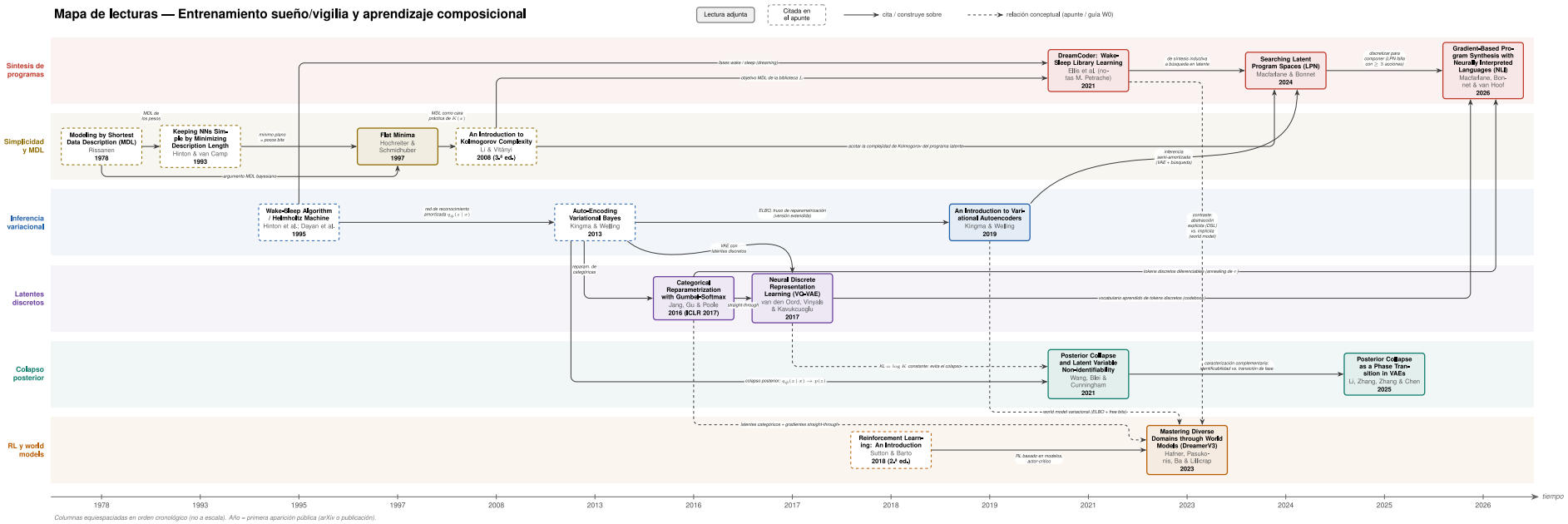
El encoder propone un programa en una pasada (Sistema 1) y la búsqueda en test-time lo refina (Sistema 2). *LPN, NLI.*

4. Piezas discretas reutilizables

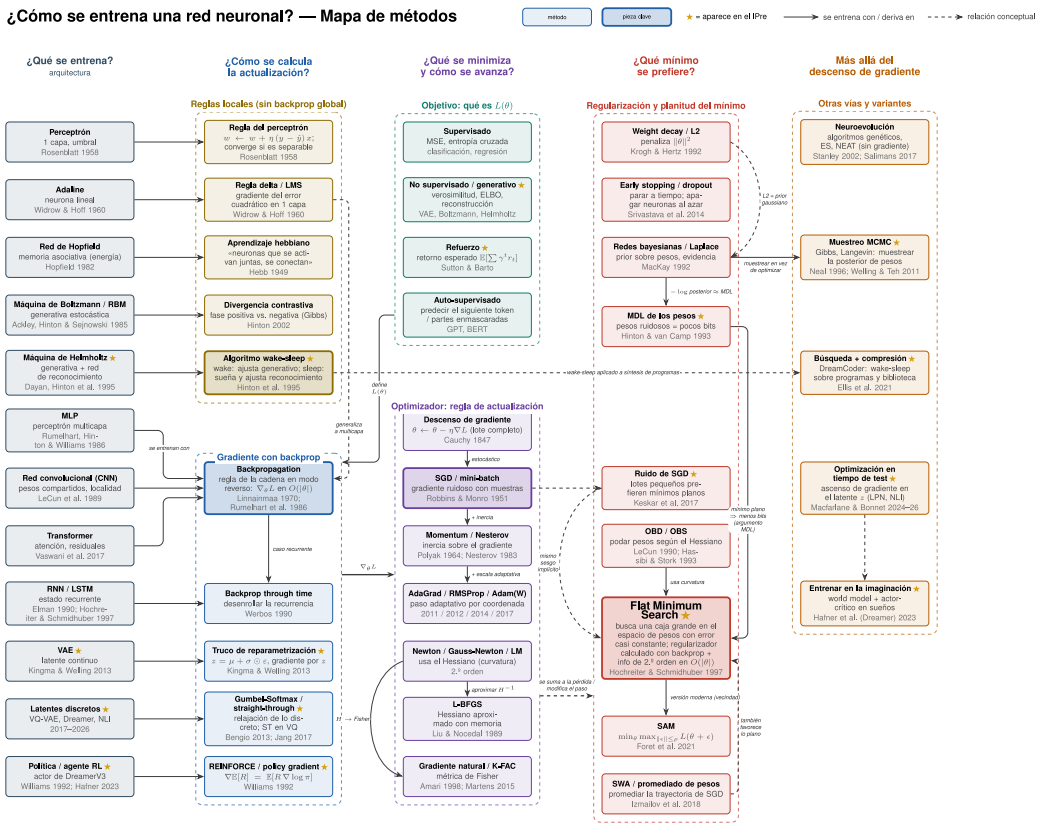
Los programas se arman con un vocabulario finito que se reutiliza y compone: la librería de *DreamCoder* y el codebook de tokens de *NLI.*

VII
Apéndice

Relaciones entre Papers



Métodos para Entrenar una Red Neuronal





IPre Sueño-Vigilia y Compositional Learning

Kevin Cardemil Ulloa
kevin.cardemil@uc.cl